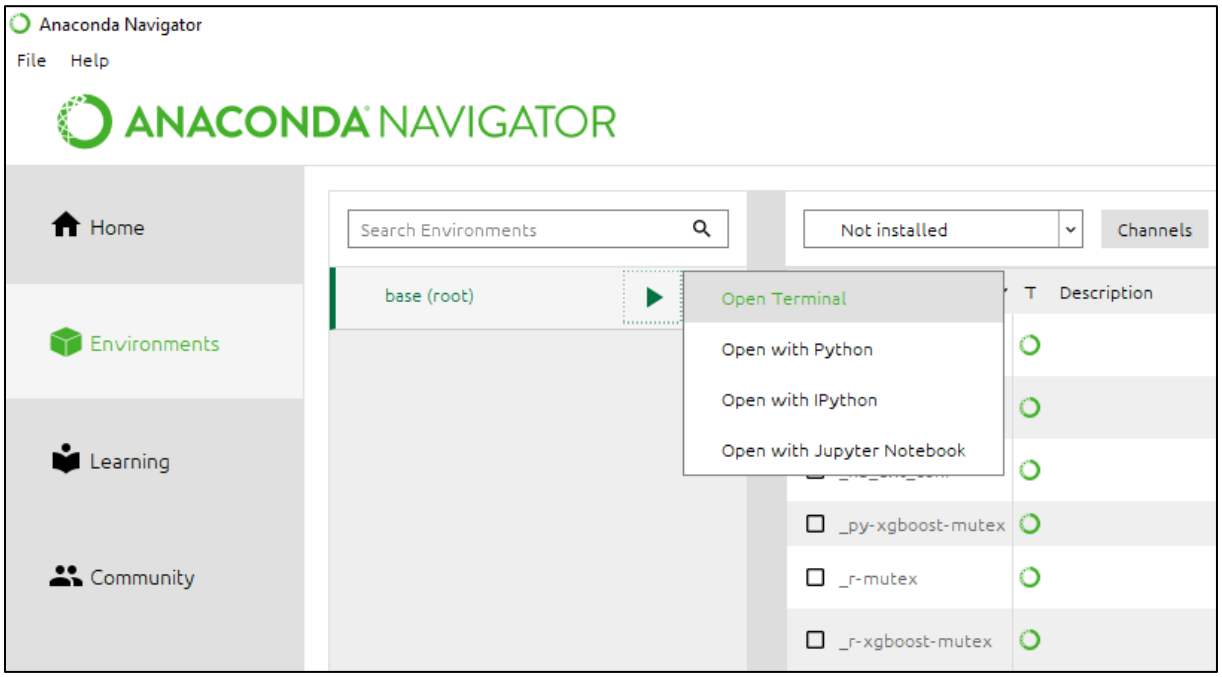
Optimization Assignment

In this exercise we will look at planning orders of plants in greenhouses.

Make sure that our optimizer PuLP is installed, by doing the following:

- Make sure that Spyder is closed.
- Start anaconda navigator.
- In the tabs to the left, click ‘Environments’. (See figure below.)
- In your base environment, click the ‘play’ button.
- Click ‘Open Terminal’.
- In the terminal that opens now, type ‘pip install pulp’ and press enter.
- On Mac/Linux you then have to type ‘pulptest’ and press enter. On Mac/Linux it may also be necessary to prefix the commands with ‘sudo’, i.e. ‘sudo pip install pulp’ and ‘sudo pulptest’

PuLP should now be installed. When it is done, you can close the terminal and start Spyder.



Exercise 1 (25 points)

A manufacturing company produces several product types using a combination of multiple raw materials, each with limited availability. Each product type uses a combination of these raw materials and can be sold for a specific profit. You are tasked to determine which products the manufacturer should produce to maximize their profit in a planning period. Note that only whole products can be produced.

Furthermore, each product requires a certain amount of time to be produced, and the total production time may not exceed 3000 time units. Lastly, due to customer contracts, the company must make at least a minimum amount of each product, as specified in the template.

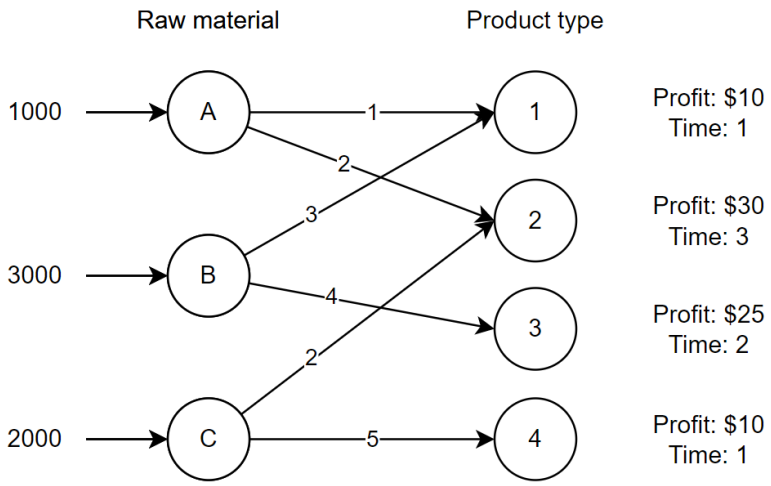
Each product has the following properties:

- A unique identifier
- The recipe, which describes the required materials and their quantities to produce one unit of a particular product type
- The number of time units required to produce one unit of a particular product type
- The minimum number of units to be produced of each product type
- The profit one unit of each product type will generate

Each material has the following properties:

- A unique identifier
- The quantity of available materials for the planning period

For example, the figure below shows an instance of the problem:



You are provided with a template file showing how the properties of the products and materials are defined as class attributes. The identifiers are defined using lists, and the other properties using dictionaries, which use the unique identifiers as keys.

You must develop your solution approach by implementing a pulp solver for the problem instance defined in the class attributes (self.material_types, self.product_types, …). Note that your solution approach should still work if those attributes are changed. Develop your solution by implementing the 'solve' function. Your solve function must return a list of tuples, where each tuple is a pair of a product type and the number of products to be produced. For example, for the data provided in the template, the following would be a well-formed result returned by

the ‘solve’ function (although it does not meet the requirements above and would therefore not be correct):

```
[('P1', 20), ('P2', 10), ('P3', 25), ('P4', 31), ('P5', 21)]
```

Note that the solution should not include products that are not produced. You can define your own helper functions.

We recommend that you follow the approach outlined in the lecture: first, determine the decision variables, then the objective function, and then each constraint. You can start playing around with the solution check and tests when you have one constraint. Then, add constraints as you see fit until you are happy with the result.

You are provided with a template file that you must complete to solve this exercise. The template file contains a basic formatting check. If this check fails, you will probably receive 0 points for this exercise. You have to test your solution by yourself.

Exercise 2 (10 points)

The goal of this exercise is the same as the previous exercise, so you can keep your 'solve' code the same. However, in this exercise, you will add a 'load' function that loads your production orders and greenhouses and their properties from text files.

The load function takes two file names. The first is the file name containing the information about the product types. The second is the file containing information about the materials. We provide you with two instances to evaluate your code. The products and materials text files are named using the following format: {exercise}_{instance}_products.txt and {exercise}_{instance}_materials.txt, respectively.

Each line in the products file represents a product type, with successively (separated by semicolons):

- the unique identifier of the product type (of type str)
- the required production time (of type float)
- the minimum quantity to be produced (of type int)
- the profit the product will make (of type int)
- the recipe (of type dict)

Each value is stored in the file as a string and may need to be converted to the correct data type. The 'load' function must load this data into the variables initialized in the constructor.

Each line in the materials file contains information about the material types, with successively:

- the unique identifier of the material type (of type str)
- the available quantity (of type int)

The 'load' function must load this data into the variables initialized in the constructor.

Format this information similarly to that in exercise 1. It should be possible to solve the loaded problem after loading.

You are provided with a template file that you must complete to solve this exercise. The template file contains a basic formatting check. If this check fails, you will probably receive 0 points for this exercise. You have to test your solution by yourself.

Exercise 3 (15 points)

In this question, we extend exercise 2 by introducing employees to the problem. Each employee is only eligible to produce a subset of all product types and produces each product type at a different speed. The production speed of an employee is modeled by a multiplier. For example, if employee ‘E1’ produces product ‘P1’ with a multiplier of 1.1, the actual production time is 1.1 * <original production time>.

Furthermore, the employees are contractors and are only available for a certain number of time units per planning period. To model these aspects, we add another argument to the ‘load’ function: efile. This file contains the following information, separated by semicolons:

- the unique identifier of the employee (of type str)
- the available time units per planning period for this employee (of type int)
- a list of tuples, where each tuple contains the product types this employee can produce and the respective production multiplier

You are provided with two problem instances (3_1 and 3_2) which you can use to evaluate your code.

The result of the solve function must now be returned as a list of triples, where the first two elements still represent which product type must be produced in what quantity. The third element represents the employee that produces the products. Note that it is now possible to have the same product type multiple times in the solution, as different employees can produce them. Similarly, one employee may produce different products. However, the result should not include multiple instances of the same employee producing the same product. For example, for the data provided in the template, the following would be a well-formed result returned by the ‘solve’ function (although it may not meet the requirements above):

[(‘P1’, 10, ‘E1), (‘P2’, 15, ‘E2’), (‘P3’, 30, ‘E1’), (‘P1’, 5, ‘E3’)]

You are provided with a template file that you must complete to solve this exercise. The template file contains a basic formatting check. If this check fails, you will probably receive 0 points for this exercise. You have to test your solution by yourself.

Exercise 4 (20 points)

Note. This exercise extends Exercise 1, not Exercise 2 or 3.

The production company is going through some hard times and must plan its operation more carefully. For this reason, we now schedule the production of each product individually for each employee.

We introduce timeslots in which an employee produces a product. The timeslots are at 1-time unit intervals, which means that orders can be placed at t=0, t=1, t=T, etc. This means there can be some slack between the production of products due to the granularity of the timeslots. For example, if we start production at t=0 and this product takes 1.5 time units to complete, then both timeslots 0 and 1 will be filled, resulting in 0.5 time units of slack, because the following product can only be started in t=2.

An employee can produce only one product at the same time, but different employees can work on different products simultaneously. For simplicity, we scale down the production plant to 5 identical employees. Each employee produces products at the same speed and is eligible to produce all product types. Furthermore, each employee is available for 20 time units in every planning period, meaning that each employee has 20 available timeslots. Note that this time limit must not be exceeded, so the current timeslot plus production time should not exceed the maximum timeslot. There is still a restriction on the total number of time units the employees can be actively working, which is 60 time units. Note that this number does not include any slack caused by planning in timeslots.

The result of the solve function must now be returned as a list of triples, where the first two elements represent which product is produced and the employee who produces it. The third element represents the timeslot in which a product of a particular type is scheduled to start for that employee. For example, for the data provided in the template, the following would be a well-formed result returned by the ‘solve’ function (although it does not meet the requirements above):

[(‘P1’, ‘E1’, ‘1’), (‘P2’, ‘E2’, ‘0’), (‘P3’, ‘E1’, ‘3’), (‘P4’, ‘E3’, ‘1’)]

You are provided with a template file that you must complete to solve this exercise. The template file contains a basic formatting check. If this check fails, you will probably receive 0 points for this exercise. You have to test your solution by yourself.

Exercise 5 (20 points)

Note. This exercise extends Exercise 1, not Exercise 2 or 3.

In this exercise, we add priority to the production of products, which indicates in what order the products should be produced. A product with a higher priority should be produced before a product with a lower priority. The priority rule is only valid for the products that a single employee produces. This means that a product of a lower priority may be produced before a product with a higher priority if another employee produces it. The priority levels are within the range [1, 2, 3, 4], where products with priority level 1 have the highest priority and should be produced first. The template shows an additional variable, *self.priority*, which defines the priority levels of each product type.

To schedule the products in order, we introduce production slots (not timeslots!) for each employee. In each production slot, a single unit of a particular product type can be produced (regardless of its production time). The production slots must be filled sequentially, such that there are no gaps between production slots. Furthermore, the first product for each employee must always be started in the first production slot.

The objective is still to optimize the total profit. Note that there is still a constraint on the total production time allowed, which still equals 60 time units.

The result of the solve function must now be returned as a list of triples, where the first two elements represent which product must be produced by which employee. The third element represents the production slot in which a product of a particular type is scheduled to start for that employee. For example, for the data provided in the template, the following would be a well-formed result returned by the 'solve' function (although it does not meet the requirements above):

[('P1', 'E1', '1'), ('P2', 'E2', '0'), ('P3', 'E1', '3'), ('P4', 'E3', '0')]

You are provided with a template file that you must complete to solve this exercise. The template file contains a self-check. Note that if the self-check fails, you will probably receive 0 points for this exercise. You are not provided with tests in this exercise. You have to test your solution by yourself.

NOTE:

- This is by all means a difficult assignment. That is also why it makes the difference between an 8 and a 10. Some of you may not be able to solve it.

HINT:

- You may want to add an additional decision variable $Y_{p,p_other,e}$ that represents that order *p_other* is in the slot immediately after order *p* for employee *e*. You can use constraint(s) to force this decision variable to 1, when this is indeed the case. You can subsequently use this variable in the objective function and further constraints.
- To speed up the code, you can use an optimizer like Gurobi. You can request a free academic license and use Gurobi as your solver.

You are provided with a template file that you must complete to solve this exercise. The template file contains a basic formatting check. If this check fails, you will probably receive 0 points for this exercise. You have to test your solution by yourself.